

Numerical Analysis of Personalized PageRank Using Variance-Reduced Monte Carlo Methods on Football Passing Networks

CSE 402 Numerical Analysis Sessional Project, Group 5, Section B2

Arian Mahtab Chowdhury
BUET
Dhaka, Bangladesh
ID: 2105106

Showkat Nur Daiyan
BUET
Dhaka, Bangladesh
ID: 2105108

Pritu Dhar Dhip
BUET
Dhaka, Bangladesh
ID: 2105109

Aveerup Chowdhury
BUET
Dhaka, Bangladesh
ID: 2105112

Abdullah Al Mahmud
BUET
Dhaka, Bangladesh
ID: 2105120

Abstract

Personalized PageRank (PPR) measures how strongly a network’s nodes are reachable from a chosen source, and it can be computed deterministically by power iteration or estimated stochastically by Monte Carlo (MC) random walks. Plain MC needs many walks because its variance is high. Liao et al. (SIGMOD 2023) reduce this variance by propagating the residual deterministically before sampling. We reimplement the paper’s estimators from scratch in Python (power iteration, standard walk MC, propagation-based walk methods PW and PPW, and loop-erased forest methods) and verify their theory on SNAP benchmark graphs. We then apply them to weighted, directed player-passing networks built from StatsBomb open event data: 100 team-match networks from four competitions. Across 30 independent trials per configuration, three damping factors ($\alpha = 0.8, 0.85, 0.99$) and four walk budgets, variance-reduced estimators cut the mean L_1 error of standard MC by roughly two orders of magnitude at equal walk budget in the moderate- α regime (for example 1.6×10^{-2} versus 7×10^{-7} for MCW versus PPW at $\alpha = 0.8$, $T = 3 \times 10^4$). Measured variances match the theoretical formulas to within a few percent in the median. The advantage shrinks, and parameter choice becomes critical, at $\alpha = 0.99$. Because the football graphs are tiny (12 to 16 players), exact solves are much faster than any sampler, so the value of these methods lies in the numerical study rather than in speed on this application.

Keywords

Personalized PageRank, Monte Carlo, variance reduction, power iteration, football passing networks

1 Introduction and Base Paper Review

PageRank [5] scores nodes by the stationary behaviour of a random surfer. Personalized PageRank (PPR) restarts the surfer from a source distribution σ instead of uniformly. With P the column-stochastic transition matrix and α the

probability of continuing a walk, the PPR vector solves

$$\pi = \alpha P\pi + (1 - \alpha)\sigma. \quad (1)$$

Convention. We use the damping-factor convention of our proposal, $\alpha =$ continue probability. The base paper’s restart probability is $1 - \alpha$, so its $\alpha = 0.2$ is our 0.8 and its hard case 0.01 is our 0.99.

Base paper. Liao et al. [4] study how to compute PPR accurately with fewer samples. Their observation is that a standard MC estimator (MCW) averages T independent α -random-walk endpoints, giving per-entry variance of order $1/T$ with a large constant. They propose three ideas. The first is *PW*. It propagates the source vector for K deterministic steps, so Monte Carlo only has to estimate the small leftover residual, which cuts the variance by a factor of about α^{2K} . The second is *PPW*, a progressive version that repeats propagate-then-sample in B batches, each working on the residual left by the previous batch. The third is a family of forest-based analogues (MCF, PF, PPF) that replace walk endpoints with roots of loop-erased-walk spanning forests [8]. The authors prove variance bounds and (ϵ, δ) relative-error guarantees, and they report large speed-ups over earlier MC methods on SNAP graphs.

Objective. Our work has three parts. In Stage 1 we reproduce the paper’s estimators and check their theory. In Stage 2 we turn football event data into weighted directed passing networks and use PPR as a measure of structural influence. In Stage 3 we run a controlled comparison of power iteration, MCW, PW, PPW and the forest methods on these networks, looking at accuracy, convergence, variance, parameter sensitivity and runtime. The football scores describe passing structure only. They do not rate players or predict results.

2 Methodology

2.1 Power iteration (deterministic baseline)

The iteration $r^{(k+1)} = \alpha P r^{(k)} + (1 - \alpha)\sigma$ contracts the L_1 error by α per step, so $\|r^{(k)} - \pi\|_1 \leq 2\alpha^k$ (a-priori bound). Given the step size $\delta_k = \|r^{(k+1)} - r^{(k)}\|_1$, the a-posteriori bound is $\|r^{(k+1)} - \pi\|_1 \leq \frac{\alpha}{1-\alpha}\delta_k$. Reaching tolerance ε needs $\approx \log \varepsilon / \log \alpha$ iterations, which grows sharply as $\alpha \rightarrow 1$. Ground truth is the direct solution of $(I - \alpha P)\pi = (1 - \alpha)\sigma$ by LU, cross-checked against power iteration at step tolerance 10^{-13} .

2.2 Standard Monte Carlo (MCW)

Each walk starts at a node drawn from σ and, at every step, stops with probability $1 - \alpha$ or moves along an out-edge with probability proportional to its weight (alias tables give $O(1)$ draws). The endpoint distribution equals π , so the histogram of T endpoints is unbiased with per-entry variance $\pi_v(1 - \pi_v)/T$, and total variance $(1 - \|\pi\|_2^2)/T$.

2.3 Variance-reduced walks (PW, PPW)

Let $x^{(0)} = \sigma$ and propagate K steps to obtain the partial sum $\hat{\pi}_K = (1 - \alpha) \sum_{k < K} \alpha^k P^k \sigma$ and residual $x^{(K)} = \alpha^K P^K \sigma$. Since $\pi = \hat{\pi}_K + \Pi x^{(K)}$ with $\Pi = (1 - \alpha)(I - \alpha P)^{-1}$, PW estimates $\Pi x^{(K)}$ by MC and adds $\hat{\pi}_K$. The residual is signed after subtractions, so walks are launched with probability proportional to $|x|$ and weighted by the sign. Variance shrinks by roughly α^{2K} (Lemma 3.7 of the paper). PPW splits the T walks into B batches. After each batch, the current estimate's residual $\sigma - (I - \alpha P)\hat{\pi}/(1 - \alpha)$ is recomputed (Lemma 3.12) and used as the next batch's source, so later batches work on ever smaller residuals.

2.4 Forest methods

MCF, PF and PPF follow Algorithm 1 and Algorithms 6 to 8 of the paper, using loop-erased α -walk forests in place of walks. Our stage-3 comparison uses only the directed variants. The degree-weighted (v) variants require undirected graphs and are excluded from football graphs.

2.5 Error metrics

We report L_1 error $\|\hat{\pi} - \pi\|_1$ (primary metric), L_2 , L_∞ , relative error, top- k overlap, and Kendall τ / Spearman ρ rank correlations, since analysts use the rankings rather than the raw scores.

3 Implementation and Experiments

3.1 Software

All algorithms are implemented from scratch in the `pprlib` Python package, using only NumPy [1] and SciPy [7] primitives (CSR sparse matrices and sparse LU). Numba is optional and only speeds up two sequential loops. Plots use Matplotlib [2]. The authors' released code was not used. Our code is linked in the Code Availability section. A pytest suite covers

Table 1: Football networks used in Stage 3 (mean, and range in parentheses).

	Match networks	Aggregates
Count	100	40
Players	14.1 (12 to 16)	19.4 (14 to 23)
Edges	105 (65 to 137)	182 (142 to 222)
Completed passes	354 (153 to 657)	726 (358 to 1236)

graph construction, estimator unbiasedness, football-network construction and Stage 3 bookkeeping.

3.2 Stage 1: benchmark reproduction

We used SNAP graphs [3]: email-Eu-core (1,005 nodes, directed), wiki-Vote (7,115 nodes, directed, with dangling nodes), ca-GrQc (5,242 nodes, undirected) and, in the full run, com-youtube (1.13M nodes). We tested the power-method rate and bounds, empirical versus theoretical variance (MCW, PW, PPW), error against walks and time, the effect of K and B , walk versus forest estimators, and the relative-error guarantee, at $\alpha = 0.8$ and 0.99 .

3.3 Stage 2: football passing networks

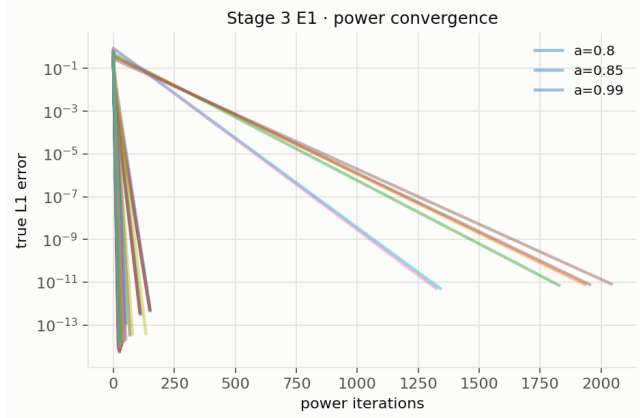
Data come from StatsBomb Open Data [6]. We selected 50 matches: 20 La Liga 2015/16, 20 Serie A 2015/16, five Indian Super League 2021/22 and five Serie A Women 2023/24, a deliberately varied convenience subset and not a random sample. Each match yields two team networks, giving **100 team-match networks** plus 40 team-season aggregates (used for Stage 2 analysis only). Nodes are players who took part in the match (starters and substitutes who came on). A directed edge passer→recipient is weighted by the count of completed passes (a pass is complete when its outcome field is absent). Match networks have 14.1 players (range 12 to 16), 105 edges and 354 completed passes on average (Table 1). Players with no outgoing passes are handled by a self-loop in P . Exact PPR was computed for every network at $\alpha \in \{0.5, 0.8, 0.85, 0.9, 0.99\}$ for every single-player source, uniform source (PageRank centrality) and positional sources (goalkeepers/defenders). Power iteration agreed with the direct solve to $L_1 \leq 9 \times 10^{-12}$ on all 13,025 checks.

3.4 Stage 3: controlled comparison design

For each of the 100 match networks we used the uniform source (PageRank centrality), $\alpha \in \{0.8, 0.85, 0.99\}$ (0.85 being the customary football value, 0.99 the hard case) and walk budgets $T \in \{10^3, 3 \times 10^3, 10^4, 3 \times 10^4\}$. Every configuration ran 30 independent seeds, and estimates were never clipped or renormalised. Timing is algorithm-only (loading, truth, metrics and plotting are excluded). Parameters were $K = 6$ for PW, and $K = 2$, $B = 3$ for PPW. The sensitivity sweep used $K \in \{0, 1, 2, 5, 10, 20, 50\}$ for PW and paired (K, B) values for PPW at a fixed $T = 3 \times 10^4$. Forest budgets were $F \in \{30, 100, 300, 1000\}$ forests. In total 5,760 configurations

Table 2: Power-iteration iterations to reach an L_1 error tolerance, over 100 networks: mean (max).

α	10^{-4}	10^{-6}	10^{-8}
0.80	8.2 (28)	13.2 (48)	18.3 (68)
0.85	9.2 (40)	15.0 (67)	20.7 (94)
0.99	43.9 (670)	67.9 (1058)	92.0 (1445)


Figure 1: Power iteration: true L_1 error versus iteration on football networks.

were summarised. A secondary cohort of min/median/max-size networks per competition serves as a robustness check for parameter and forest sweeps. The root seed was 202603, and the complete run takes about 1.5 hours on a laptop.

4 Results and Discussion

4.1 Power iteration as baseline (E1)

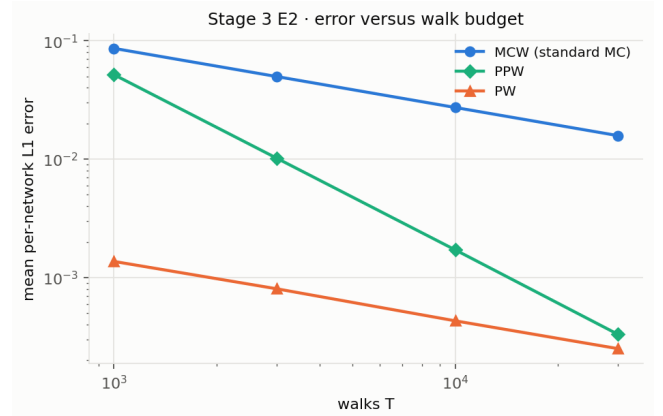
Across all networks the observed contraction matches theory, and both the a-priori and a-posteriori bounds held at every iteration in our records. Iterations to reach a tolerance grow steeply with α (Table 2). At $\alpha = 0.99$ the mean is 92 iterations for 10^{-8} but the worst network needs 1,445. Stage 1 shows the same on SNAP graphs, where the empirical rate lies just below α (for instance 0.777 versus 0.8 on email-Eu-core), and the a-priori bound is loose for high α (521 measured iterations versus 1,902 predicted for 10^{-8} at $\alpha = 0.99$). Because $n \approx 14$, a sparse or dense exact solve costs about 10^{-4} s, less than any iterative or sampling method: in this application exact computation is the sensible choice, and the estimators are studied as numerical methods.

4.2 Error versus walk budget (E2)

Table 3 and Fig. 2 give the mean L_1 error over 100 networks. MCW decays slowly and roughly as $T^{-1/2}$: a 30-fold increase in walks lowers the error from 8.8×10^{-2} only to 1.6×10^{-2} . At $\alpha = 0.8$ both variance-reduced methods are far better at the same T : PW is about 140 $\times$ better at $T = 10^3$, and PPW about 750 $\times$ better at $T = 10^3$ and $2 \times 10^4 \times$ at $T = 3 \times 10^4$.

Table 3: Mean L_1 error (100 match networks, 30 trials, uniform source) and median algorithm time in ms.

α	Method	$T=10^3$	$T=3 \times 10^4$	top-10 @ 10^3	ms @ 3×10^4
0.80	MCW	8.8×10^{-2}	1.6×10^{-2}	0.89	7.5
	PW	6.3×10^{-4}	1.1×10^{-4}	1.00	7.7
	PPW	1.2×10^{-4}	7.3×10^{-7}	1.00	9.1
0.99	MCW	8.5×10^{-2}	1.6×10^{-2}	0.91	164
	PW	2.6×10^{-3}	4.7×10^{-4}	1.00	163
	PPW	1.6×10^{-1}	1.0×10^{-3}	0.93	188


Figure 2: Mean L_1 error versus number of walks T (averaged over α and networks).

PPW's error falls much faster with T than PW's, because each extra batch works on an ever smaller residual, whereas PW is limited by its fixed K . Consequently PW wins at small budgets in the hard regime, and PPW overtakes it at large budgets in the easy regime. At $\alpha = 0.99$ the picture differs: PPW with $K = 2$ is *worse* than MCW at $T = 10^3$ (0.163 versus 0.085) and only overtakes it from $T = 3 \times 10^3$. PW remains best throughout (about 33 $\times$ better than MCW at $T = 10^3$). Rankings follow the same pattern: MCW's top- k overlap with the exact ranking stays at 0.89 to 0.97 even at $\alpha = 0.8$, while PW/PPW recover the top- k set essentially perfectly. Note that the mean over α in the figure hides PPW beating PW at large T .

4.3 Theory versus experiment (E3)

For MCW and PW we compared the total sample variance over 30 trials with the theoretical variance (Lemma 3.2 and the exact PW variance) in 1,632 configurations each. The ratio empirical/theory has median 0.994 (interquartile range 0.94 to 1.05) for MCW and median 0.970 (0.84 to 1.12) for PW. The spread is about what 30-trial variance estimates give, and Fig. 3 shows empirical and theoretical curves on top of each other at all three α . The empirical variance for PW sits about 4% below the closed form on median, so we consider the theory confirmed but do not claim exactness. PPW variance is not tabulated in Stage 3, but Stage 1 tests it

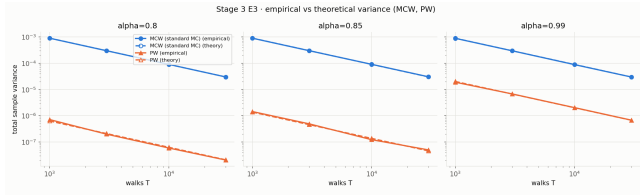


Figure 3: Empirical (solid) versus theoretical (dashed) variance of MCW and PW.

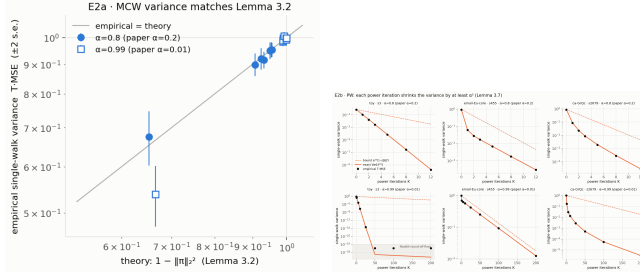


Figure 4: Stage 1 (SNAP graphs): variance validation for MCW (left) and PW as a function of K (right).

per batch on SNAP graphs. Stage 1 gave the same conclusion for MCW and PW variance (Fig. 4). For example on a toy graph MCW variance is 0.675 ± 0.036 against theory 0.653.

4.4 Choosing K and B (E4)

PW’s error decreases steadily with K (median L_1 over networks: 1.6×10^{-2} at $K = 0$, 4×10^{-3} at $K = 1$, 3×10^{-5} at $K = 5$, 8×10^{-16} at $K = 50$ for $\alpha = 0.8$), the runtime being nearly unchanged since K propagation steps on a 14-node graph are trivial. PPW is more delicate (Fig. 5). At $\alpha = 0.8$ the error is best at small K with more batches ($K = 1$, $B = 10$: 5×10^{-13}), while at $\alpha = 0.99$ the same setting is unusable: its median error is 1.9, worse than doing nothing. With continuation probability 0.99, a one-step propagation leaves a residual of mass close to 1 whose signed contributions accumulate across batches, so the residual grows instead of shrinking. Larger K ($K = 10$, $B = 1$) gives 10^{-6} . The mean-error curve in Fig. 5, aggregated over α , therefore shows a “sweet spot” shape that is really a mixture of these two regimes. The practical rule is: for α near 1, use K of the order of $\log(10^{-4})/\log \alpha$ rather than a small value. Table 3’s PPW result at $\alpha = 0.99$ uses $K = 2$ and so is a deliberately unfavourable setting, not a tuned one.

4.5 Runtime and accuracy trade-off, and forests (E5)

Fig. 6 plots error against algorithm-only time for every configuration. Methods with propagation (PW, PPW) and the forest estimators PF/PPF occupy the lower-left region (low error, low time). Standard MCW and MCF are 2 to 4 orders of magnitude less accurate at the same time. At equal walk

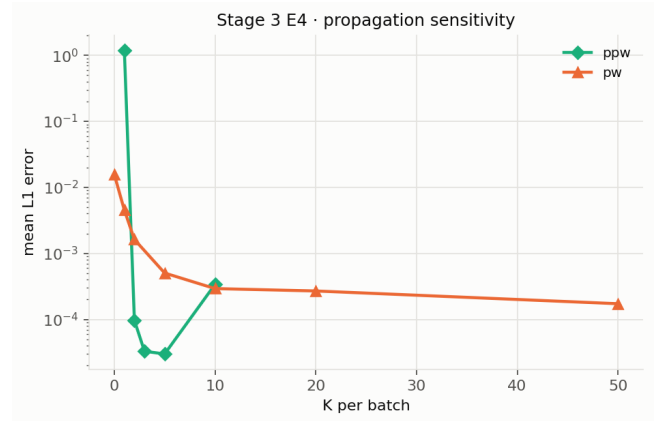


Figure 5: Sensitivity to the per-batch propagation length K ($T = 3 \times 10^4$, mean over α , networks and trials).

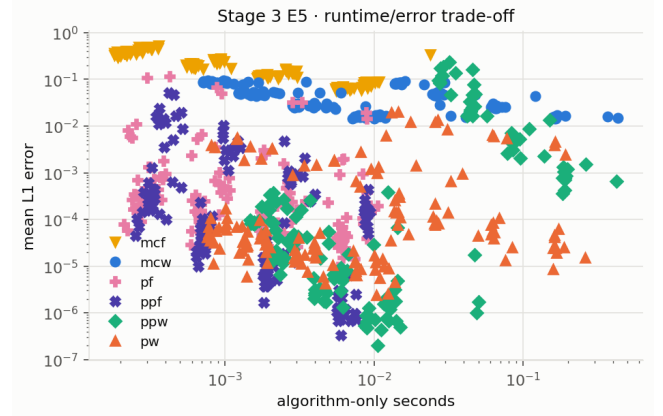


Figure 6: Runtime versus mean L_1 error over all configurations, by method.

count, PW costs the same as MCW (median 163 versus 164 ms at $\alpha = 0.99$, $T = 3 \times 10^4$), so the variance reduction is nearly free. PPW costs 10 to 20% more. Forest methods take about 6 ms for 1,000 forests. Among forests, PF/PPF beat MCF by a wide margin: at $\alpha = 0.8$ and $F = 1000$, mean L_1 is 3×10^{-5} (PF), $< 10^{-5}$ (PPF) versus 6×10^{-2} (MCF). Reaching a target error of 10^{-3} was possible for PPF in all 36 tested cases, for PW/PPW/PF in 32 of 36, and never for MCW or MCF within the tested budgets. We only count crossings that happened inside the tested range.

4.6 Comparison with the base paper

Our findings agree with the paper qualitatively: the propagated estimators are dramatically more accurate than plain MC at equal walk budget, and high α is hard for everything. They also differ in scale and in setting. The paper targets million-node graphs with time-to-accuracy as the goal, while our graphs have about 14 nodes, where power iteration and

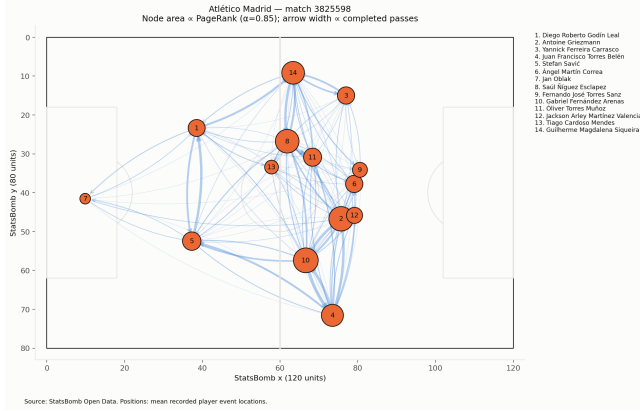


Figure 7: Passing network for one team-match (node area $\propto$ PageRank, edge width $\propto$ passes).

direct solves dominate and the sampling overhead cannot pay off. On the SNAP benchmarks we did reproduce the variance lemmas, the power-method bounds and the effect of K and B (Stage 1), and the relative-error experiment with theory-derived T and K met its guarantee in every recorded run (176 of 176 for MCW, PW and PPW). In the Stage 1 walk-versus-forest comparison on the three SNAP graphs, the forest estimators had a larger MSE than the walk estimator in all six cases (two α values each), which is what the Lemma 4.4 criterion predicted for those graphs and settings.

4.7 Football-specific observations

Exact centrality rankings are stable to the choice of α : Kendall τ against $\alpha = 0.85$ averages 0.99 at $\alpha = 0.8$, 0.99 at 0.9, 0.97 at 0.99 and 0.92 at 0.5 (across all networks). This means the football conclusions are not fragile with respect to this parameter, whereas the difficulty of *computing* PPR at $\alpha = 0.99$ is much larger. An example team network appears in Fig. 7.

5 Conclusion and Future Work

We reimplemented the variance-reduced Monte Carlo PPR estimators of Liao et al. and compared them with power iteration and standard MC on 100 football passing networks. Deterministic propagation before sampling gave large accuracy gains at equal walk budget at almost no extra cost, measured variances agreed with the theory, and progressive batching (PPW) achieved the lowest errors at moderate α . At $\alpha = 0.99$, the parameters K and B must be chosen with care or the progressive method can be worse than plain MC.

Limitations. (i) The networks are tiny, so exact methods beat every estimator on speed. Speed conclusions do not transfer to large graphs, which we tested only in Stage 1. (ii) The 50 matches are a convenience sample from four competitions. (iii) The main Stage 3 cohort uses the uniform source and one parameter setting per method apart from the

sweeps, and PPW at $\alpha = 0.99$ was not tuned. (iv) Some timing measurements have outliers (a single PPW configuration at $\alpha = 0.99$, $T = 10^4$ averaged 7.9s, likely a system artefact), so we report medians. (v) Our figure-level averages mix α regimes. (vi) PPR measures structural influence in passing, not player skill.

Future work. Natural next steps are choosing K and B automatically from α , testing larger networks such as aggregated seasons or event-level graphs where sampling should pay off, studying in-match dynamic networks with warm-started estimates, and putting bootstrap uncertainty on the rankings.

Code Availability

The full implementation is in a public GitHub repository: <https://github.com/Arian892/CSE-402-PROJECT>. It contains the `pplib` estimator library, the football network code, the Stage 1 to 3 experiment scripts, the tests, and a README with setup and run instructions.

References

- [1] Charles R. Harris et al. 2020. Array programming with NumPy. *Nature* 585 (2020).
- [2] John D. Hunter. 2007. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering* 9, 3 (2007).
- [3] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. <http://snap.stanford.edu/data>.
- [4] Meihao Liao, Rong-Hua Li, Qiangqiang Dai, Hongyang Chen, Hongchao Qin, and Guoren Wang. 2023. Efficient Personalized PageRank Computation: The Power of Variance-Reduced Monte Carlo Approaches. *Proceedings of the ACM on Management of Data (SIGMOD)* 1, 2 (2023).
- [5] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. 1999. *The PageRank Citation Ranking: Bringing Order to the Web*. Technical Report. Stanford InfoLab.
- [6] StatsBomb. [n. d.]. StatsBomb Open Data. <https://github.com/statsbomb/open-data>. Accessed 2026.
- [7] Pauli Virtanen et al. 2020. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods* 17 (2020).
- [8] David Bruce Wilson. 1996. Generating Random Spanning Trees More Quickly than the Cover Time. In *Proc. 28th ACM Symposium on Theory of Computing (STOC)*.